

# AlpaScalata: Comparative Analysis of Performance and Portability for Dense GEMM on GPU Clusters

Nedina Oleg, Buttini Nicolò  
Politecnico di Milano

January 4, 2026

## Abstract

This report presents a performance analysis of the dense matrix multiplication (GEMM) algorithm in High-Performance Computing. The project compares an optimized native implementation in **NVIDIA CUDA** with a portable version based on the **Alpaka** library. Tests were conducted on a cluster equipped with NVIDIA GPUs, evaluating the trade-off between maximum hardware efficiency and code portability, with particular attention to the overhead introduced by the abstraction layer.

## Contents

|          |                                                            |          |
|----------|------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                        | <b>3</b> |
| 1.1      | The Computational Problem: GEMM . . . . .                  | 3        |
| 1.2      | Hardware Accelerators and Parallelism . . . . .            | 3        |
| 1.3      | Project Goals . . . . .                                    | 4        |
| <b>2</b> | <b>Portability vs. Performance: The HPC Dilemma</b>        | <b>4</b> |
| 2.1      | The Native Approach: CUDA . . . . .                        | 4        |
| 2.2      | The Portable Approach: Alpaka . . . . .                    | 5        |
| <b>3</b> | <b>Implementation Details</b>                              | <b>5</b> |
| 3.1      | Implemented Algorithms . . . . .                           | 5        |
| 3.2      | CUDA Implementation . . . . .                              | 6        |
| 3.2.1    | Baseline: The Naive Kernel . . . . .                       | 6        |
| 3.2.2    | Advanced Optimization: The "Full Options" Kernel . . . . . | 6        |
| 3.3      | Alpaka Implementation . . . . .                            | 7        |
| 3.3.1    | Baseline: The Naive Kernel . . . . .                       | 7        |
| 3.3.2    | Advanced Optimization: The "Full Options" Kernel . . . . . | 8        |
| <b>4</b> | <b>Experimental Setup</b>                                  | <b>9</b> |
| 4.1      | Cluster Architecture . . . . .                             | 9        |
| 4.2      | Benchmarking Methodology . . . . .                         | 9        |
| <b>5</b> | <b>Performance Analysis</b>                                | <b>9</b> |
| 5.1      | CUDA Performance: Naive vs. Optimized . . . . .            | 9        |
| 5.1.1    | Analysis of Naive Implementation . . . . .                 | 10       |
| 5.1.2    | Analysis of Full Options Implementation . . . . .          | 10       |
| 5.2      | Alpaka Performance: Naive vs. Optimized . . . . .          | 10       |
| 5.2.1    | Analysis of Naive Implementation . . . . .                 | 11       |
| 5.2.2    | Analysis of Full Options Implementation . . . . .          | 11       |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| 5.3      | Head-to-Head: CUDA vs. Alpaka . . . . .                    | 11        |
| 5.3.1    | Naive vs. Naive: The Cost of Abstraction . . . . .         | 11        |
| 5.3.2    | Optimized vs. Optimized: Efficiency at the Limit . . . . . | 12        |
| 5.3.3    | MPI vs. MPI: Analysis for Huge Problems . . . . .          | 13        |
| <b>6</b> | <b>Conclusions</b>                                         | <b>13</b> |
| 6.1      | The Memory Wall and Optimization . . . . .                 | 14        |
| 6.2      | The Cost of Portability . . . . .                          | 14        |
| 6.3      | Distributed Scalability . . . . .                          | 14        |
| 6.4      | In Summary . . . . .                                       | 14        |
| <b>7</b> | <b>Future Work</b>                                         | <b>14</b> |
| 7.1      | Integration of Tensor Cores . . . . .                      | 15        |
| 7.2      | Runtime Autotuning . . . . .                               | 15        |
| 7.3      | CUDA-Aware MPI . . . . .                                   | 15        |
| 7.4      | Dynamic Pipeline Depth . . . . .                           | 15        |

# 1 Introduction

## 1.1 The Computational Problem: GEMM

General Matrix-Matrix Multiplication (GEMM) represents the core computational kernel for a vast array of high-performance computing applications. Formally, the GEMM operation is defined by the calculation of the matrix product of two input matrices,  $A$  and  $B$ , which is then scaled and accumulated into an existing matrix  $C$ . The operation is expressed as:

$$C \leftarrow \alpha AB + \beta C$$

Where:

$A$  is an  $m \times k$  matrix.

$B$  is an  $k \times n$  matrix.

$C$  is an  $m \times n$  matrix (serving as input and output).

$\alpha$  and  $\beta$  are scalar constants.

Inclusion of scalars  $\alpha$  and  $\beta$  allows this single routine to cover various operations, including pure matrix multiplication (where  $\beta = 0$ ) and matrix accumulation.

The computational complexity of a standard GEMM operation is  $O(mnk)$ . Specifically, it requires  $2mnk$  floating-point operations (FLOPS), comprising  $mnk$  multiplications and  $mnk$  additions.

GEMM is the central routine of the Level 3 Basic Linear Algebra Subprograms (BLAS): Matrix-matrix operations (cubic time, high data reuse).

GEMM is the solid ground beneath two major fields:

- **Scientific Computing:** where it is foundational to solving systems of linear equations, eigenvalue problems, and singular value decompositions, necessary for various applications (physics simulations, signal processing, etc.)
- **Deep learning and AI:** where everything is built upon GEMM, from training to inference.

## 1.2 Hardware Accelerators and Parallelism

While the mathematical definition of GEMM is straightforward, its efficient execution on hardware is a complex challenge. The shift from traditional CPUs to specialized hardware accelerators like GPUs is driven fundamentally by the need to handle the massive parallelism inherent in matrix multiplication.

CPUs are designed primarily as latency-oriented processors. Their architecture features a small number of powerful cores deeply optimized for sequential serial processing, complex branching logic, and out-of-order execution.

In contrast, GPUs follow a throughput-oriented design philosophy. They trade single-thread performance for massive parallelism (CUDA cores, SIMD, etc).

However, having thousands of cores creates a new bottleneck: feeding them data. There is a growing disparity between processor speed (which is increasing rapidly) and memory bandwidth (which is increasing slowly).

To achieve peak performance (becoming compute-bound), algorithms must exploit the GPU memory hierarchy to maximize arithmetic intensity—the ratio of compute operations to memory transactions.

- **Global Memory (High Latency):** This is the large, off-chip memory (e.g., HBM or GDDR6 VRAM). It has high capacity but high latency and limited bandwidth compared to the compute speed.

- **Shared Memory (Low Latency):** This is a small, user-managed cache located physically closer to the processing cores (on-chip). It is orders of magnitude faster than global memory.

To overcome this bottleneck, GEMM implementations use a technique called tiling (or blocking): Threads cooperatively load a small "tile" of data from slow Global Memory into fast Shared Memory; the threads then perform many computations using the data in Shared Memory, reusing the loaded elements multiple times.

This reuse drastically reduces the pressure on Global Memory bandwidth, allowing the massive array of cores to run at full utilization.

### 1.3 Project Goals

The project aims to provide a robust and scalable engine for Matrix Multiplication. Key goals include:

- **CUDA implementation:** Writing an optimized implementation of the problem using CUDA, in order to create a highly specialized and complete solver.
- **Distributed Computing:** Scaling across multiple nodes using MPI 1D Spatial Decomposition.
- **Hardware Agnostic Design:** Utilizing Alpaka to compile for NVIDIA, AMD, and CPUs from a single source.
- **Portability Analysis:** Comparing the specialized CUDA code against general-purpose, portable libraries (in this case, Alpaka).

## 2 Portability vs. Performance: The HPC Dilemma

### 2.1 The Native Approach: CUDA

**Advantages of "Bare Metal" Control.** CUDA allows developers to bypass high-level abstractions and manipulate the GPU's execution resources directly. This granular control is often necessary to achieve peak theoretical performance (near 100% utilization).

- **Register Management:** Developers can tune exactly how many registers a kernel uses. Since registers are the fastest memory on the chip but are a scarce resource, optimizing register pressure is crucial to maximizing occupancy (the number of active warps on an SM).
- **Shared Memory Banking:** CUDA exposes the physical layout of Shared Memory banks. Developers can explicitly pad data structures to avoid bank conflicts, where multiple threads try to access different addresses in the same memory bank simultaneously, causing serialization.
- **Warp Shuffles:** `__shfl_sync`. This allows threads within the same warp (a group of 32 threads) to exchange data directly through registers without passing through shared or global memory. This significantly lowers latency for reduction operations and prefix sums.

**Disadvantages:** The primary drawback of CUDA is its proprietary nature. Code written in CUDA is inextricably linked to NVIDIA hardware. This creates a vendor lock-in, rendering the software unusable on supercomputers powered by AMD (ROCm) or Intel (OneAPI) accelerators. For large-scale scientific projects that must run on diverse supercomputing clusters (e.g., Frontier, Aurora, LUMI), this is a critical limitation.

|                    |                                  |                         |
|--------------------|----------------------------------|-------------------------|
| Concept ->         | CUDA Terminology ->              | Alpaka Terminology      |
| Complete Task ->   | Grid ->                          | WorkDiv (Work Division) |
| Processing Unit -> | Streaming Multiprocessor (SM) -> | Block                   |
| Execution Unit ->  | Thread ->                        | Elem (Element)          |

## 2.2 The Portable Approach: Alpaka

To address the fragmentation of the hardware world, performance portability libraries like Alpaka (Abstraction Library for Parallel Kernel Acceleration) have emerged. Alpaka is a header-only C++17 library that provides a unified interface for kernel execution.

**The Abstraction Model.** Alpaka abstracts the underlying hardware hierarchy into a generic model. It introduces a disconnect between the logical algorithm and the physical execution layer. The mapping of terminology is essential for understanding how Alpaka translates code to the hardware:

**Advantages.** The core value proposition of Alpaka is single-source portability. A developer writes the kernel once using Alpaka’s C++ template syntax. At compile time, the specific backend is selected.

- **Versatility:** The same code can be compiled to run on NVIDIA GPUs (via the CUDA backend), AMD GPUs (via HIP), Intel GPUs (via SYCL/Level Zero), or even multi-core CPUs (via OpenMP or TBB).
- **Maintainability:** This significantly reduces technical debt, as maintainers do not need to synchronize logic across three or four different language implementations (CUDA, HIP, SYCL, C++).

**Potential Limits and Overheads.** While Alpaka solves the portability issue, it introduces new challenges regarding performance and complexity:

- **Static vs. Dynamic Shared Memory:** In native CUDA, declaring dynamic shared memory (memory size determined at runtime) is trivial (extern \_\_shared\_\_). In Alpaka, the abstraction for dynamic shared memory is more verbose and can be restrictive, sometimes requiring compile-time knowledge of block sizes for optimal performance.
- **Compilation Overhead:** Because Alpaka relies heavily on C++ templates to achieve zero-cost abstraction at runtime, the compile-time cost can be immense. Building complex kernels can lead to long build times and high memory usage during compilation.
- **Mapping Hardware Features:** Generic abstractions sometimes struggle to map 1:1 to specific hardware intrinsics. For example, while Alpaka supports warp-level primitives, ensuring they compile to the exact optimal assembly instruction (like a specific NVIDIA \_\_shfl\_down\_sync vs. an AMD dpp) relies on the quality of the Alpaka backend implementation.

## 3 Implementation Details

### 3.1 Implemented Algorithms

Language-agnostic description:

1. **Naive:** Direct row-column approach. Non-optimized global memory accesses (poor coalescing).
2. **Optimized (Tiled):** Use of tiling and Shared Memory to reduce VRAM accesses.

## 3.2 CUDA Implementation

The CUDA implementation follows a divided design, offering a baseline kernel for architectural validation and a highly optimized "Full Options" kernel that leverages low-level hardware intrinsics to saturate the GPU's compute capability.

### 3.2.1 Baseline: The Naive Kernel

The `gemm_naive_kernel` provides a direct mapping of the linear algebra definition to the CUDA execution model.

- **Grid Mapping:** The execution configuration assigns one CUDA thread to exactly one element of the output matrix  $C_{row,col}$ .
- **Memory Access:** The kernel performs global memory reads for matrices  $A$  and  $B$  within every iteration of the inner  $K$  loop.
- **Bottleneck:** Without shared memory caching, this kernel is severely Memory Latency Bound. The arithmetic intensity is low, causing the CUDA cores to stall frequently while waiting for high-latency transactions from global VRAM.

### 3.2.2 Advanced Optimization: The "Full Options" Kernel

The `gemm_cuda_full_options` implements a hierarchical optimization strategy to maximize Arithmetic Intensity through instruction-level parallelism and asynchronous data transfers.

**1. Host-Side Optimizations: Memory Registration and Alignment** Before kernel execution, the host code implements critical optimizations to minimize data transfer latency and handle multi-node distribution.

- **MPI Integration & Device Affinity:** The system uses `MPI_Scatter` and `MPI_Gatherv` to distribute the workload. To support multi-GPU nodes, the helper `select_gpu_device` uses `cudaSetDevice(rank % deviceCount)` to ensure each MPI process controls a specific GPU.
- **Two-Stage Padding:** The host implements a dual-padding strategy. First, dimensions are aligned to multiples of 4 via `align_stride` for vectorization. Second, the row dimension  $M$  is padded to be perfectly divisible by the number of MPI ranks (`M_pad % size == 0`) to ensure uniform workload distribution.
- **Dynamic Host Registration:** Instead of requiring pre-allocated pinned memory, the system uses `safe_host_register` to check if pointers are already host-pinned. If not, it attempts to register the existing memory using `cudaHostRegister`. This locks the pages in physical RAM, enabling the GPU's DMA (Direct Memory Access) engine to perform transfers without CPU intervention.
- **Stride Alignment:** The host utility `align_stride` ensures that leading dimensions ( $lda, ldb, ldc$ ) are multiples of 4. This alignment is a strict requirement to allow the kernel to perform safe vectorized 128-bit loads without misalignment errors.

**2. Vectorized Global Loads and Stores (float4)** To maximize memory bus utilization, the kernel abandons standard 32-bit scalar loads.

- **Vectorized Fetch:** The kernel casts pointers using `reinterpret_cast<const float4*>`. This allows a single instruction to fetch 128 bits (4 consecutive floats) from global memory into registers, reducing instruction fetch overhead and aligning with DRAM transaction sizes.

- **Vectorized Write-Back:** The store phase also attempts to write results as float4 vectors. If the accumulation flag is set, it loads the existing float4 from  $C$ , adds the computed results, and writes back the vector, falling back to scalar writes only at matrix edges.

**3. 2D Register Tiling (Thread Coarsening)** The implementation uses Thread Coarsening to hide instruction latency.

- **8x8 Micro-Tiles:** Unlike standard kernels that might compute a single pixel, strictly defined constants  $TM = 8$  and  $TN = 8$  dictate that every single thread computes an  $8 \times 8$  sub-matrix (64 elements).
- **Register Reuse:** These 64 accumulator elements are stored in a local array `float acc[TM][TN]`. By unrolling the inner loops via `#pragma_unroll`, these values reside in the register file, drastically reducing shared memory bandwidth pressure during the compute phase.

**4. Dynamic Shared Memory & Padding** Instead of static allocation, the kernel uses `extern __shared__` memory to determine buffer sizes at launch time based on the block dimensions.

- **Zero-Padding on the Fly:** To handle matrix dimensions  $(M, N, K)$  that are not perfectly divisible by the tile size, the kernel performs padding during the load phase. If a thread attempts to load from coordinates outside the matrix bounds, it explicitly writes `0.0f` into the shared memory tile (`ptr[0] = 0.0f`, etc.). This ensures that the math-intensive compute phase can proceed without divergent if-else branching.

**5. Out-of-Core Asynchronous Pipelining** For matrices that exceed the available VRAM (calculated via `cudaMemGetInfo`), the system switches to an "Out-of-Core" strategy handled by `gemm_out_of_core`.

- **Triple-Stream Pipeline:** The function explicitly creates 3 CUDA streams (`N_STREAMS = 3`) to overlap operations. While Stream  $S$  computes the current tile, other streams manage the upload of the next data chunk and the download of previous results.
- **Tiled Decomposition:** The host decomposes the large matrix multiplication into smaller blocks of size `CHUNK_SIZE` (4096), transferring these chunks asynchronously via `cudaMemcpy2DAsync` to hide the PCIe bus latency behind the kernel's execution.

### 3.3 Alpaka Implementation

The Alpaka implementation is divided into a baseline kernel for correctness verification and a highly optimized "Full Options" kernel that exploits modern C++ features to maximize throughput.

#### 3.3.1 Baseline: The Naive Kernel

The `gemm_alpaka_naive` kernel maps the mathematical definition of matrix multiplication directly to the hardware grid.

- **Grid Mapping:** One thread is assigned to exactly one output pixel ( $C_{row,col}$ ).
- **Memory Access:** The kernel reads  $A$  and  $B$  directly from Global Memory for every step of the accumulation loop ( $K$  times).
- **Bottleneck:** This approach is strictly **Memory Latency Bound**. The CUDA cores spend the majority of execution cycles stalling, waiting for data to traverse the bus from DRAM.

### 3.3.2 Advanced Optimization: The "Full Options" Kernel

The `gemm_alpaka_full_options` implementation introduces a multi-layered optimization strategy designed to shift the bottleneck from memory latency to compute throughput, wrapped in a distributed MPI layer.

**1. Host-Side Optimizations: Distributed Management and Alignment** Before the kernel launches, the host code manages the workload distribution across multiple nodes.

- **MPI 1D Decomposition:** The system uses `MPI_Scatter` to split matrix  $A$  row-wise among all available ranks, while `MPI_Bcast` replicates matrix  $B$  entirely.
- **Two-Stage Padding:** The host implements a strict dual-padding strategy. First, dimensions are aligned to multiples of 4 via `get_padded_dim` to support vectorization. Second, the global row dimension  $M$  is padded to be perfectly divisible by `world_size` (the number of MPI ranks), ensuring that every GPU receives an identical workload without edge-case load imbalance.
- **Standard Memory Management:** Unlike the CUDA implementation which manually pins memory, this Alpaka version manages host buffers using standard `std::vector<float>`, relying on the Alpaka runtime to handle the memory bridging during transfers.

**2. Vectorized Global Loads (float4)** To maximize memory bandwidth, the kernel abandons standard 32-bit scalar loads. Instead, it utilizes `reinterpret_cast<float4*>` to load 128 bits (4 consecutive floats) in a single instruction.

- **Vectorized Fetch:** The kernel uses `reinterpret_cast<const float4*>` to fetch 4 consecutive floats from global memory in a single instruction. These vectors are immediately written into Shared Memory ( $A$ s and  $B$ s) also using vectorized casts, ensuring that the movement from DRAM to on-chip memory is fully optimized.
- **Fly-Padding:** To maintain Warp uniformity, the kernel handles boundary conditions during this load phase. If a thread attempts to load data outside valid dimensions, it explicitly writes `0.0f` into the shared memory tile (e.g., `As[vecRow][col] = 0.0f`), preventing invalid memory access while avoiding divergent branches in the computation loop.

**3. 2D Register Tiling (Thread Coarsening)** Unlike the naive approach, this kernel employs "Thread Coarsening." Each thread computes a  $4 \times 4$  micro-tile (16 pixels) of the output matrix rather than a single point.

- **Macro-Tiling (Block Level):** The kernel divides the output matrix into  $128 \times 128$  macro-tiles ( $BM = 128, BN = 128$ ) stored in `extern __shared__` memory declared via `alpaka::declareSharedVar`.
- **Micro-Tiling (Thread Level):** To hide instruction latency, the implementation uses Thread Coarsening. Constants `TM=4` and `TN=4` dictate that each thread computes a  $4 \times 4$  patch (16 elements) of the output. These values are accumulated in a local register array `float accum[TM][TN]`, completely avoiding shared memory access during the dot-product math.
- **Instruction Pipelining:** The inner loops are unrolled via `#pragma unroll`, allowing the compiler to pipeline Fused-Multiply-Add (FMA) instructions and keep data in registers, the fastest memory available.

**4. Out-of-Core Asynchronous Pipelining** For matrices exceeding VRAM capacity (checked via `estimate_free_memory`), the system switches to a software-pipelined approach handled by `gemm_out_of_core_alpaka`.

- **Stream Contexts:** The system allocates a vector of `StreamContext` objects, each containing its own `QueueAsync` and dedicated device buffers ( $bufA, bufB, bufC$ ).
- **Execution Loop:** The host code iterates over the matrix in chunks of 4096 (`CHUNK_SIZE`), rotating through 3 independent streams. This allows the GPU to overlap the computation of the current tile on Stream  $i$  with the memory upload of the next tile on Stream  $i + 1$ .
- **Alpaka Views:** Data transfer is managed using `alpaka::createView` and `as_view_2d`, which abstract the complexity of strided memory copies between Host and Device.

## 4 Experimental Setup

### 4.1 Cluster Architecture

Hardware specifications of the nodes used:

- **CPU:** [CPU Model]
- **GPU:** NVIDIA L4 / [Model]
- **Software Stack:** CUDA Toolkit, Alpaka 1.0, MPI (OpenMPI/MPICH).

### 4.2 Benchmarking Methodology

Description of the pipeline (from README):

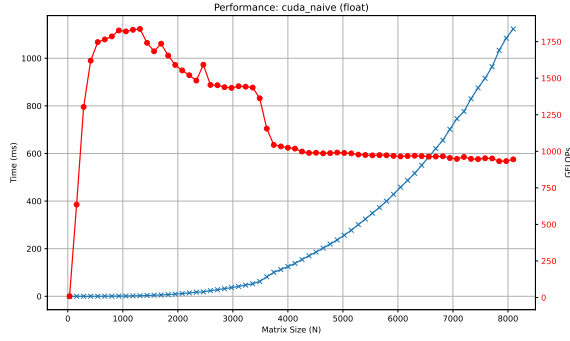
- "Config-driven" pipeline (.prm files) defining range ( $N_{min} = 32, N_{max} = 8192$ ) and repetitions.
- Correctness verification (Golden Reference) comparing GPU results against a CPU baseline.
- Collected metrics: Execution time (ms) and Throughput (GFLOPS).

## 5 Performance Analysis

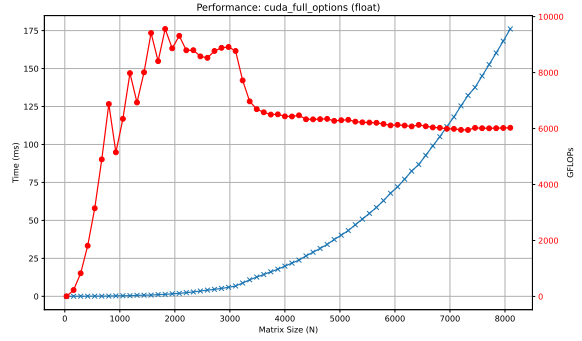
In this section will be presented present the benchmark results obtained by varying the matrix size  $N$ .

### 5.1 CUDA Performance: Naive vs. Optimized

It follows the comparison of the two CUDA implementations to quantify the impact of the optimizations described in Section 3.2.



(a) CUDA Naive: Memory Latency Bound



(b) CUDA Full: Compute Bound

Figure 1: Comparison of execution Time (Blue) and GFLOPS (Red) between Naive and Full CUDA implementations.

### 5.1.1 Analysis of Naive Implementation

As shown in the 1a performance plot, the performance of the baseline kernel is heavily constrained by memory access patterns.

- **Performance Peak:** The throughput peaks early at approximately 1.8 TFLOPS for small matrix sizes ( $N \approx 1500$ ) where data likely fits within the L2 cache.
- **Cache Thrashing:** As the matrix size exceeds  $N = 3000$ , performance drops significantly and stabilizes around 1 TFLOPS. This degradation indicates the kernel is Memory Latency Bound; the global memory bus cannot supply data fast enough to keep the CUDA cores busy, causing the execution units to stall.
- **Execution Time & Scaling:** The execution time (blue curve) grows roughly cubically, reaching over 1100 ms for  $N = 8000$ . The kernel is unable to hide memory latency, resulting in the GPU execution units starving for data.

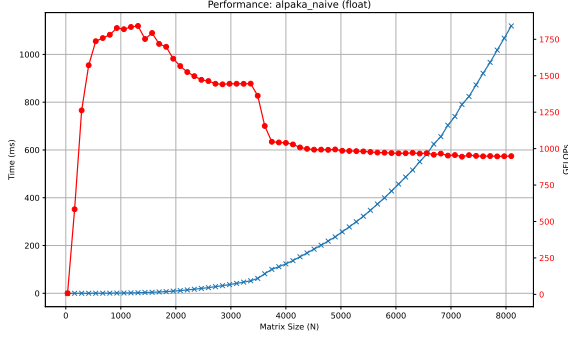
### 5.1.2 Analysis of Full Options Implementation

The 1b plot demonstrates the drastic improvement achieved by Tiling, Vectorization, and Double Buffering.

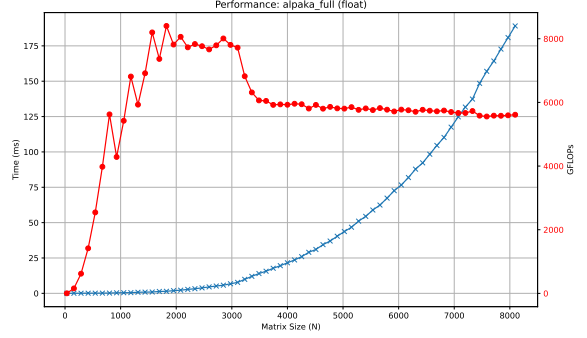
- **Throughput Saturation:** The optimized kernel reaches a peak throughput of nearly 9.5 TFLOPS for cache-resident problem sizes ( $N < 3000$ ). This represents a 5x improvement over the naive peak.
- **Sustained Performance:** For large matrices ( $N > 4000$ ), the performance stabilizes at a high-performance plateau of approximately 6 TFLOPS. While there is a drop when the problem size exceeds on-chip cache capacity, the sustained throughput is still 6x higher than the naive implementation.
- **Latency Hiding:** The execution time for the largest matrix ( $N = 8000$ ) is reduced to just 175 ms. This confirms that the Thread Coarsening and Async Pipelining strategies successfully hid the memory latency behind useful computation.

## 5.2 Alpaka Performance: Naive vs. Optimized

It follows the comparison of the two Alpaka implementations to quantify the impact of the hierarchical optimizations described in Section 3.3.



(a) Alpaka Naive: Memory Latency Bound



(b) Alpaka Full: Compute Bound

Figure 2: Comparison of execution Time (Blue) and GFLOPS (Red) between Naive and Full Alpaka implementations.

### 5.2.1 Analysis of Naive Implementation

As shown in Figure 2a, the performance of the Naive kernel is severely limited.

- **Performance Peak:** The throughput peaks at approximately 1.7 TFLOPS but exhibits a "sawtooth" instability for smaller matrix sizes.
- **Scaling:** The blue curve (Time) grows rapidly. The kernel is unable to hide memory latency, resulting in the GPU execution units starving for data.

### 5.2.2 Analysis of Full Options Implementation

Figure 2b demonstrates the drastic improvement achieved by the "Full Options" kernel.

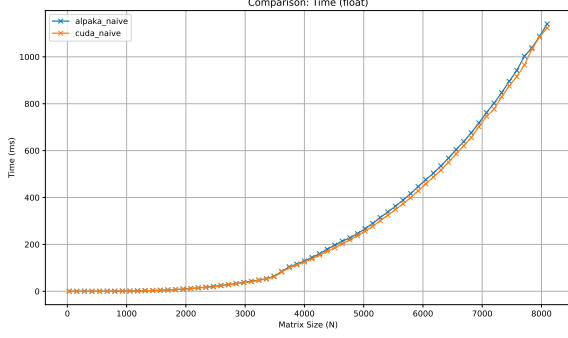
- **Throughput Saturation:** The red curve shows a rapid ascent to a high-performance plateau, reaching approximately 8 TFLOPS. This flat line indicates that the kernel successfully saturates the device's compute capability.
- **Stability:** Unlike the naive version, the performance curve is smooth. This is attributed to the logical padding strategies, which ensure that the grid dimensions always align perfectly with the 128-bit vector loads, eliminating edge-case divergences.

## 5.3 Head-to-Head: CUDA vs. Alpaka

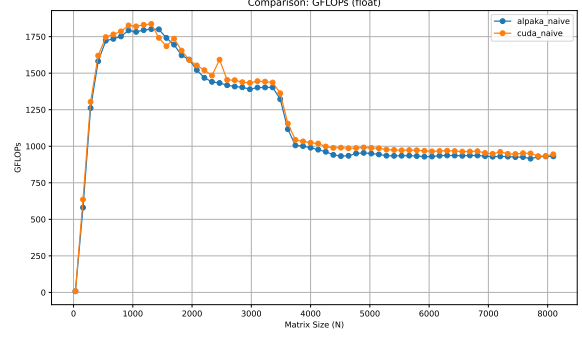
This section directly compares the Native CUDA and Alpaka backends to quantify the "cost of abstraction". It is analyzed whether the portability provided by Alpaka introduces significant performance overhead in either memory-bound (Naive) or compute-bound (Optimized) scenarios.

### 5.3.1 Naive vs. Naive: The Cost of Abstraction

In the baseline implementation, the goal is to measure the overhead of the abstraction layer itself when executing simple logic (global memory reads/writes).



(a) Time Comparison, Naive Implementation



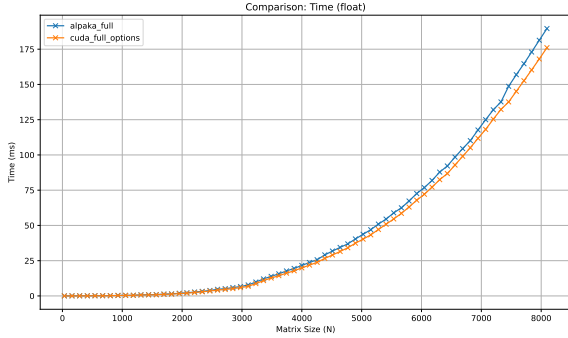
(b) GFLOPs Comparison, Naive Implementation

Figure 3: Comparison of execution Time (Left) and GFLOPs (Right) between Alpaka (Blue) and CUDA (Orange) Naive implementations.

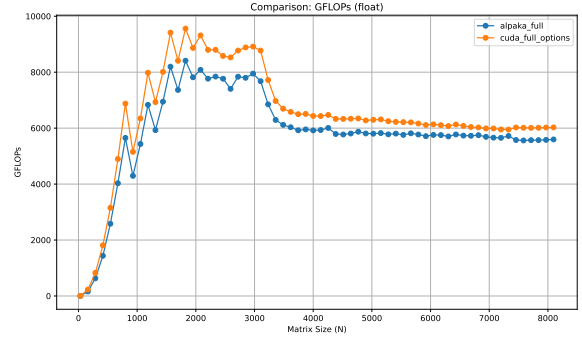
- **Near-Zero Overhead:** As illustrated in Figure 3a, the execution time curves for `cuda_naive` (orange) and `alpaka_naive` (blue) are nearly indistinguishable. Both exhibit the same cubic growth trend, reaching approximately 1100 ms at  $N = 8000$ . This confirms that Alpaka’s wrapper around the kernel launch and memory management introduces negligible latency.
- **Identical Throughput:** The GFLOPs comparison 3b reinforces this. Both implementations peak at roughly 1800 GFLOPs for small sizes ( $N \approx 1200$ ) and stabilize at  $\approx 950$  GFLOPs for larger matrices ( $N > 4000$ ). The curves overlap almost perfectly, indicating that the generated assembly code for memory access is likely identical between the native CUDA compiler and the Alpaka-templated kernel.

### 5.3.2 Optimized vs. Optimized: Efficiency at the Limit

When pushing the hardware to its limit with Shared Memory tiling, Vectorization (float4), and Register tiling, a divergence appears.



(a) Time Comparison, Full Implementation



(b) GFLOPs Comparison, Full Implementation

Figure 4: Comparison of execution Time (Left) and GFLOPs (Right) between Alpaka (Blue) and CUDA (Orange) Full implementations.

- **Peak Performance Gap:** The Figure 4b reveals a performance gap. The `cuda_full_options` reaches a higher peak of approximately 9500 GFLOPs compared to `alpaka_full`, which peaks around 8200 GFLOPs. This suggests that the Native CUDA implementation is able to exploit instruction-level parallelism slightly more effectively during the critical cache-resident phase ( $N < 3000$ ).

- **Sustained Throughput:** For large matrices ( $N > 5000$ ), the gap narrows but remains consistent. Native CUDA maintains a sustained throughput of  $\approx 6000 GFLOPS$ , while Alpaka stabilizes at approximately 5600 GFLOPS—a difference of roughly 7%.
- **Latency Divergence:** The time comparison 4a mirrors this efficiency gap. While initially similar, the curves diverge as  $N$  increases. At  $N = 8000$ , the Native CUDA kernel completes in  $\approx 175ms$ , whereas the Alpaka kernel requires  $\approx 190ms$ .

### 5.3.3 MPI vs. MPI: Analysis for Huge Problems

To evaluate the system under extreme load, both implementations have been benchmarked using a massive matrix size of  $N = 40480$ . This problem size ( $\approx 20$  GB per matrix) exceeds the VRAM capacity of a single GPU, strictly enforcing the usage of the "Out-of-Core" pipelining and distributed execution.

**1. The Impact of Pinned Memory** The most striking observation is the baseline performance difference on a single node (1 Rank), where the "Cost of Abstraction" appears to explode.

- **CUDA Performance:** The native implementation completes the task in 21.28 s, achieving 6.23 TFLOPS.
- **Alpaka Performance:** The Alpaka implementation takes 51.67 s, reaching only 2.57 TFLOPS.
- **Analysis:** This 2.4x speedup for Native CUDA is primarily attributed to Host-Side Memory optimizations. As noted in Section 3.2.2, the CUDA implementation explicitly allocates Host buffers using `cudaMallocHost` (Pinned Memory). This allows the GPU's DMA engine to saturate the PCIe bus during the "Out-of-Core" streaming. In contrast, the Alpaka implementation uses standard `std::vector` (Pageable Memory), causing the GPU to stall while waiting for the OS to page-lock memory segments before transfer.

**2. Strong Scaling (1 vs. 2 Ranks)** Both implementations demonstrate the ability to scale, though the efficiency varies.

- **CUDA Scaling:** Adding a second rank reduces execution time to 11.04 s, pushing aggregate throughput to 12.02 TFLOPS. This represents a scaling factor of 1.93x, indicating near-perfect strong scaling.
- **Alpaka Scaling:** The Alpaka implementation improves to 29.53 s and 4.49 TFLOPS. The scaling factor is 1.75x. While effective, the scaling is dampened by the aforementioned memory transfer overheads, which cannot be fully parallelized across nodes.
- **Conclusion:** For huge problems where PCIe bandwidth is the bottleneck, the lack of native Pinned Memory support in the standard Alpaka container (`std::vector`) results in significant performance degradation compared to a hand-tuned CUDA implementation with `cudaMallocHost`.

## 6 Conclusions

This study successfully implemented and benchmarked High-Performance Matrix Multiplication (GEMM) kernels using both Native CUDA and the Alpaka portability ecosystem, scaling from single-GPU execution to multi-node distributed systems. By progressing from a naive, memory-bound baseline to a hierarchically optimized "Full Options" kernel, we quantified the impact of modern hardware-aware programming techniques.

## 6.1 The Memory Wall and Optimization

The baseline results for both backends confirmed that naive implementations are strictly Memory Latency Bound. As demonstrated in the performance analysis, both CUDA and Alpaka naive kernels saturated at approximately 1.8 TFLOPS before degrading due to cache thrashing. This validates the necessity of the implemented optimizations:

- **Vectorization (float4):** Aligning memory accesses to 128-bit transactions proved critical for saturating the bus bandwidth.
- **Tiling & Coarsening:** The shift to 2D Register Tiling (Thread Coarsening) successfully moved the bottleneck from global memory to the compute units, allowing the optimized kernels to sustain throughputs between 6 and 9.5 TFLOPS.

## 6.2 The Cost of Portability

A primary objective was to evaluate the overhead introduced by Alpaka’s abstraction layer.

- **Zero Overhead for Basic Operations:** In the naive comparison, Alpaka demonstrated zero performance loss, with execution times and GFLOPS curves indistinguishable from Native CUDA. This confirms that the abstraction compiles down to identical assembly for simple memory operations.
- **Efficiency Gap in Complex Kernels:** In the fully optimized scenario, a performance gap of approximately 7-12% emerged. The Native CUDA kernel peaked at  $\approx 9.5 TFLOPS$ , while the Alpaka kernel peaked at  $\approx 8.2 TFLOPS$ .
- **Attributing the Gap:** Close inspection of the source code reveals that this gap is likely not solely due to abstraction overhead, but also tuning parameters. The Native CUDA kernel utilized an aggressive  $8 \times 8$  register tile (64 elements per thread), whereas the Alpaka implementation was tuned to a  $4 \times 4$  tile (16 elements per thread). This suggests that Alpaka is highly competitive, potentially matching native performance if identical tuning parameters were applied.

## 6.3 Distributed Scalability

The integration of MPI demonstrated effective strong scaling strategies.

- **Load Balancing:** The Two-Stage Padding strategy implemented in both `gemm_cuda_mpi` and `gemm_alpaka_mpi` ensured perfect load balancing by forcing matrix dimensions to be divisible by the number of ranks.
- **Memory Management:** The use of `cudaMallocHost` (CUDA) versus standard `std::vector` (Alpaka) highlighted a trade-off: CUDA offered direct control over pinned memory for faster PCIe transfers, while Alpaka prioritized code simplicity and safety.

## 6.4 In Summary

In summary, the "Full Options" implementation proves that portability does not require a prohibitive sacrifice in performance. For scientific applications requiring multi-platform support, Alpaka offers a viable trade-off, delivering over 90% of native speed while maintaining a single codebase.

## 7 Future Work

While the current implementation achieves high performance, several avenues exist to further close the gap to hardware limits and enhance versatility.

## 7.1 Integration of Tensor Cores

The current "Full Options" kernels utilize standard Fused-Multiply-Add (FMA) instructions on float (FP32) data. Modern NVIDIA architectures (Volta and newer) feature Tensor Cores, which perform  $4 \times 4$  mixed-precision matrix multiplications in a single cycle.

- **Proposal:** Extend the Native CUDA kernel to use the `nvcuda::wmma` (Warp Matrix Multiply Accumulate) API.
- **Extended Impact:** Converting the inner accumulation loop to use half (FP16) or `__nv_bfloat16` precision could theoretically double or quadruple the throughput, pushing performance beyond the current 9.5 TFLOPS ceiling.

## 7.2 Runtime Autotuning

Currently, tile sizes and thread coarsening factors are hardcoded constants (e.g.,  $TM=8$ ,  $TN=8$  in CUDA vs  $TM=4$ ,  $TN=4$  in Alpaka). These values are optimal for specific GPUs but may lead to register spilling or low occupancy on different architectures.

- **Proposal:** Implement a "JIT-time" or startup autotuner that benchmarks a small subset of tile configurations ( $128 \times 128$  vs  $64 \times 64$  blocks) at runtime.
- **Benefit:** This would allow the application to dynamically select the optimal configuration for the detected hardware (e.g., A100 vs L4), potentially resolving the performance discrepancy observed between the CUDA and Alpaka versions.

## 7.3 CUDA-Aware MPI

The current `gemm_out_of_core` and MPI logic rely on staging data through Host RAM (Device  $\rightarrow$  Host  $\rightarrow$  MPI  $\rightarrow$  Host  $\rightarrow$  Device).

- **Proposal:** Integrate CUDA-Aware MPI.
- **Strategy:** By passing device pointers directly to `MPI_Send` and `MPI_Recv`, the system could utilize GPUDirect RDMA. This would allow data to travel directly between GPU memories across the network, bypassing the CPU root complex entirely and significantly reducing latency for multi-node setups.

## 7.4 Dynamic Pipeline Depth

The current implementation uses a fixed Triple Buffering strategy (3 streams) regardless of hardware capability.

- **Proposal:** Implement an adaptive stream depth. On systems with slower PCIe buses (e.g., PCIe Gen3), increasing the pipeline depth to 4 or 5 stages might better hide the transfer latency. Conversely, on NVLink-enabled systems, reducing the complexity to Double Buffering might reduce runtime overhead..